

Swapping Nodes in a Linked List

Authored by: Arpit Raj | Institution: LNMIIT Jaipur | Date: June 24, 2026

Problem Definition: Given the head of a linked list and an integer k , swap the values of the k^{th} node from the beginning and the k^{th} node from the end (the list is 1-indexed).

💡 KEY ALGORITHMIC INSIGHT

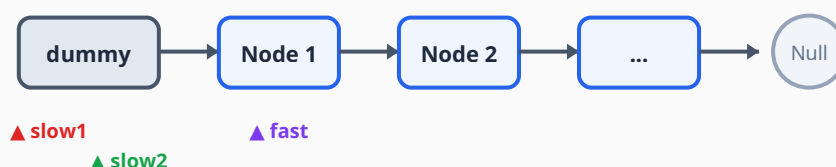
Whenever you see a requirement involving the k^{th} node from the end, always think of the **Fixed Gap Pattern** (Two Pointers/Sliding Window technique for linked lists).

Approach 1: Three-Pointer / Fixed Gap Pattern (with Dummy Node)

This approach utilizes a dummy node to cleanly handle edge cases and establishes a fixed distance gap of k between pointers to locate the target elements efficiently.

- S1** Create a **Dummy node** and link it to the head.
- S2** Create and initialize the **slow** and **fast** pointers.
- S3** Create a fixed gap of k nodes between **slow** and **fast**. When **fast** reaches the end (**NULL**), **slow1** will perfectly reach the k^{th} node from the end.
- S4** Create another slow pointer (**slow2**) and advance it to find the k^{th} node from the beginning.
- S5** **Swap** the values of the two identified nodes (**slow1** and **slow2**).

VISUAL LAYOUT & POINTER TRACKING



```
// S1: Advance fast pointer to create a gap of k nodes
for (int i = 0; i < k; i++) {
```

```

    fast = fast->next;
} // Time Taken: O(k)

// S2: Move fast to the end; slow1 will then point to kth node from end
while (fast) {
    slow1 = slow1->next;
    fast = fast->next;
} // Time Taken: O(n - k)

// S3: Move slow2 pointer to find the kth node from the beginning
while (k--) {
    slow2 = slow2->next;
} // Time Taken: O(k)

// S4: Swapping the values of slow1 and slow2
temp = slow1->val;
slow1->val = slow2->val;
slow2->val = temp; // Values swapped successfully

```

Total Time Complexity (TC):

$O(k) + O(n - k) + O(k) = O(2k + n) \rightarrow O(n)$

Space Complexity (SC):

$O(1)$ (Constant auxiliary space)

Approach 2: One-Pass Solution (Alternate Optimization)

An optimization can be performed over Approach 1 to restrict node traversals strictly to a single pass by reusing the position of the first target node.

- S1** Move **slow2** first to directly reach the k^{th} node from the beginning.
- S2** Initialize the **fast** pointer directly from **slow2** itself. This single-pass optimization achieves a crisp traversal layout.

Optimized Metrics:

TC = $O(n)$ | SC = $O(1)$

Approach 3: Length-Based Approach

A direct alternative strategy involves computing the complete length of the linked list first, then navigating to the explicit indices from the head.

- S1** Find the total length of the linked list (**len**).
- S2** Advance the **first** pointer to the k^{th} node from the beginning.
- S3** Advance the **second** pointer to the k^{th} node from the end (position layout: $len - k + 1$).
- S4** Swap the node values using standard pointer variable swapping.

```
// S1: Find length (implicitly done prior)

// S2: Traverse to the kth node from beginning
for (int i = 0; i < k; i++) {
    first = first->next;
}

// S3: Traverse to the kth node from the end
for (int i = 0; i < (len - k + 1); i++) {
    second = second->next;
}

// S4: Swap values
swap(first->val, second->val);
```

Time Complexity (TC):

$O(2 * n) \rightarrow O(n)$

Space Complexity (SC):

$O(1)$